Stable Diffusion from Scratch

A Decision-Maker's Guide

Lecture Notes

CAS BMAI — ETH Zürich

Format:	5 interactive notebooks, approx. 2.5 hours total
Prerequisites:	None (ability to click "Run" in Google Colab)
Audience:	Managers and non-technical leaders

April 2, 2026

Contents

1 Introduction **3**
- 1.1 The Co-Pilot Metaphor . . . 3
- 1.2 Toy Dataset . . . 3

2 Notebook 1: The Raw Ingredients—Images and Text as Numbers **3**
- 2.1 Cold Open: The End Product . . . 4
- 2.2 What Computers See: Images as Tensors . . . 4
- 2.3 Words as Numbers: Embedding Vectors . . . 4
- 2.4 Productive Failure: The Random Projection . . . 4

3 Notebook 2: The Translator—CLIP **5**
- 3.1 The Core Idea: Contrastive Learning . . . 5
- 3.2 Watching CLIP Learn . . . 5
- 3.3 Building Mini-CLIP . . . 5
- 3.4 Productive Failure: Memorisation vs. Generalisation . . . 5
- 3.5 The Fix: Partial Matches Reveal Concept Learning . . . 6

4 Notebook 3: The Engine—Diffusion **6**
- 4.1 Forward Process: Adding Noise . . . 6
- 4.2 Reverse Process: Denoising . . . 6
- 4.3 The Cosine Noise Schedule . . . 7
- 4.4 Noise Prediction Loss . . . 7
- 4.5 Productive Failure: The Naïve MLP . . . 7

5 Notebook 4: The Conversation—Cross-Attention **8**
- 5.1 From Unconditional to Conditional Generation . . . 8
- 5.2 The MicroUNet Architecture . . . 8
- 5.3 How Cross-Attention Works . . . 8
- 5.4 Demo: Cross-Attention ON vs. OFF . . . 8
- 5.5 Visualising Attention Heatmaps . . . 9
- 5.6 Interactive Explorer: Guidance Scale . . . 9
- 5.7 Productive Failure: Uniform Attention . . . 9

6 Notebook 5: The Road Trip—The Full Pipeline **9**
- 6.1 Assembling the Pipeline . . . 9
- 6.2 Classifier-Free Guidance . . . 10
- 6.3 Interactive Prompt Lab . . . 10
- 6.4 Productive Failure: Compositional Complexity . . . 10
- 6.5 Safety: Where Would Filters Go? . . . 10
- 6.6 Bridge to Real Tools . . . 10

7 The Co-Pilot Reference Card **11**

8 Five Questions to Ask an AI Image Generation Vendor **11**

9 Vendor Evaluation: Decision Tree **12**

10 Jargon Decoder **13**

11 Red Flags When Evaluating Vendors **13**

12 Conclusion **14**

1 Introduction

Text-to-image generation—the ability to produce photorealistic pictures from a short textual description—has moved from a research curiosity to a strategic capability in less than three years. Products such as DALL-E, Midjourney, and Stable Diffusion now power content pipelines in marketing, e-commerce, entertainment, and product design.

These lecture notes accompany a hands-on course in which participants *build* a miniature text-to-image system themselves, step by step. The system is small enough to train on a laptop (approximately 300 K parameters) yet architecturally identical to the full-scale models used in production. The goal is not to turn managers into machine-learning engineers but to equip them with a *working mental model* of the technology—one precise enough to evaluate vendors, diagnose failures, and ask engineering teams the right questions.

1.1 The Co-Pilot Metaphor

Throughout the course we use a single, consistent analogy that maps every technical component to a familiar concept:

Technical Component	Metaphor
Diffusion model (U-Net)	The *driver*—knows how to navigate but not where to go
Text prompt	The *destination*
CLIP text encoder	The *co-pilot* reading the map
Cross-attention	The driver asking the co-pilot at every turn
Guidance scale	How *loudly* the co-pilot speaks
Negative prompt	The co-pilot saying "NOT that way"
Random seed	The random road network

By the end of Notebook 5 you will have assembled all parts of this "car" and driven it yourself.

1.2 Toy Dataset

All experiments use a **programmatically generated** dataset of coloured geometric shapes on a 32×32 pixel canvas:

- **Shapes:** circle, square, triangle
- **Colours:** red, blue, green
- **Base dataset:** 9 classes (3 shapes × 3 colours)
- **Extended dataset (Notebook 5):** 36 classes (adding small/large and centred/offset)

Because every image is generated with known ground truth, we can measure model quality objectively at every stage—an advantage rarely available with real-world data.

2 Notebook 1: The Raw Ingredients—Images and Text as Numbers

Approx. 25 minutes. Metaphor: "Here are the parts of the car and the map."

2.1 Cold Open: The End Product

The notebook opens with a striking demonstration: a pre-trained model generates coloured shapes from two-word prompts such as "red circle" or "blue square." Participants see the finished product *before* learning how it works—establishing a concrete target for the sessions ahead.

2.2 What Computers See: Images as Tensors

A digital image is stored as a three-dimensional grid of numbers—a **tensor** of shape $H \times W \times C$, where H is the height in pixels, W the width, and C the number of colour channels (three for RGB). Each entry is a value between 0 and 1 representing the intensity of one colour at one pixel location.

In the interactive exercise, participants adjust **brightness** and **contrast** sliders and watch the raw numbers update in real time, building intuition for how visual changes correspond to numerical changes.

2.3 Words as Numbers: Embedding Vectors

Computers cannot process words directly. Instead, every word is mapped to a list of numbers called an **embedding vector**. Words with similar meanings end up close together in this numerical space; unrelated words end up far apart.

The notebook visualises nine word embeddings (colours and shapes) projected onto a two-dimensional plane, letting participants see that "red," "blue," and "green" cluster together, while "circle," "square," and "triangle" form a separate cluster.

The notion of "closeness" is quantified by **cosine similarity**: two vectors pointing in roughly the same direction score near 1; orthogonal vectors score 0.

2.4 Productive Failure: The Random Projection

Participants attempt to compare raw pixel vectors to text embeddings using a naïve random projection. The resulting similarity matrix shows a random pattern with no meaningful structure.

Common Pitfall

Image pixels and word embeddings live in completely different numerical spaces. A random projection cannot bridge them—the similarity matrix looks like noise. Bridging this gap requires *joint training*, as we will see in Notebook 2.

Manager's Briefing

Resolution = Cost. Our toy images are $32 \times 32 = 3{,}072$ numbers. Stable Diffusion operates at $512 \times 512 = 786{,}432$ numbers; SDXL at $1024 \times 1024 = 3{,}145{,}728$ numbers. Higher resolution means proportionally more computation and cost.
Vendor question: "What resolution does your model generate natively?"

Manager's Briefing

Joint vs. Post-Hoc Alignment. One could train an image system and a text system separately, then bolt them together after the fact (*post-hoc* alignment). As the random-projection failure demonstrates, this does not work. Effective systems train text and image encoders *jointly* from the start—the approach taken by CLIP.

Vendor question: "Was your text encoder trained jointly with your image model?"

3 Notebook 2: The Translator—CLIP

Approx. 30 minutes. Metaphor: "The co-pilot learns to read the map."

3.1 The Core Idea: Contrastive Learning

CLIP (Contrastive Language–Image Pre-training) learns to place images and their corresponding text descriptions *near each other* in a shared numerical space, while pushing mismatched pairs apart.

Training proceeds as follows:

1. Collect a batch of image–text pairs.
2. Pass each image through an **image encoder** to obtain an image embedding.
3. Pass each text through a **text encoder** to obtain a text embedding.
4. Compute the cosine similarity between every image embedding and every text embedding in the batch.
5. Adjust the encoders so that *matching* pairs score high and *non-matching* pairs score low.

The loss function that drives this adjustment is called **InfoNCE** (with a temperature parameter $\tau = 0.07$). It is computed symmetrically: text-to-image and image-to-text, averaged together.

Key Insight

After training, CLIP provides a *common language* for images and text. You can compare an image to a sentence—or two images, or two sentences—simply by measuring the cosine similarity of their embedding vectors.

3.2 Watching CLIP Learn

An interactive **epoch slider** (0 to 100) shows a PCA projection of both text embeddings (orange circles) and image embeddings (blue squares). At epoch 0, matching pairs are scattered randomly. As training progresses, matching text and image embeddings migrate toward each other until, by epoch 100, all nine pairs are tightly aligned.

3.3 Building Mini-CLIP

Participants inspect two small neural networks:

- **Text encoder:** word embedding (vocabulary of 12 tokens, 32 dimensions) $\rightarrow$ mean pooling $\rightarrow$ two linear layers with ReLU $\rightarrow$ L2 normalisation.
- **Image encoder:** flatten $32 \times 32 \times 3 = 3{,}072$ pixels $\rightarrow$ two linear layers with ReLU $\rightarrow$ L2 normalisation.

The complete Mini-CLIP model has approximately 200K parameters and trains in $\sim$15 seconds on a CPU.

3.4 Productive Failure: Memorisation vs. Generalisation

When Mini-CLIP is trained on only **three** image–text pairs (red circle, blue square, green triangle), it memorises them perfectly—the similarity matrix shows three bright spots on the diagonal. But it fails completely on the other six classes: the off-diagonal entries are noise.

The model has *memorised* individual pairs rather than learning general concepts ("redness," "circularity"). Only when all nine pairs are used does the model generalise.

3.5 The Fix: Partial Matches Reveal Concept Learning

With all nine training pairs, Mini-CLIP learns transferable concepts. The text "red square" has moderate similarity to the image of a "red circle" (shared colour), and "blue circle" scores well against "green circle" (shared shape). This *partial-match* structure proves that the model learned colour and shape as independent features—not just rote associations.

Real-World Story: The CLIP Bias Problem

Researchers discovered that CLIP embeddings encode societal biases present in web-scraped training data. Searching for "CEO" retrieves predominantly white male faces; "criminal" retrieves disproportionately Black men. The diagnosis: contrastive learning faithfully encodes correlations in the data—including unwanted ones.

Manager's Briefing

Fine-Tuning CLIP.

- **Too few pairs** → memorisation.
- **Enough pairs** → concept learning.
- **Massive corpus** → rich, transferable knowledge.

Vendor questions: How large is the training set? Has demographic bias been tested? How do you prevent catastrophic forgetting when fine-tuning?

Manager's Briefing

Practical Tool: Bias Auditing. You can audit a CLIP model by computing cosine similarity between a concept embedding (e.g., "doctor," "engineer," "nurse") and embeddings of demographically varied images. High variance in similarity scores across demographics signals bias encoded in the model.

4 Notebook 3: The Engine—Diffusion

Approx. 35 minutes. Metaphor: "The driver learns to navigate by being dropped at random locations."

4.1 Forward Process: Adding Noise

The diffusion framework rests on a surprisingly simple idea. Start with a clean image and gradually add random noise, step by step, until the image becomes pure static. This is the **forward process**—it is easy, deterministic given the noise schedule, and requires no learning.

4.2 Reverse Process: Denoising

The creative act is running this process *in reverse*: starting from pure noise and removing noise step by step until a clean image emerges. A neural network—the **denoiser**—is trained to predict the noise that was added at each step. Given a noisy image and the current step number, it outputs an estimate of the noise, which is then subtracted.

Repeating this subtraction across all steps (50 in the demo) transforms random static into a coherent image.

4.3 The Cosine Noise Schedule

The rate at which noise is added is controlled by a **noise schedule**. The course uses a **cosine schedule**, which adds noise slowly at first, accelerates in the middle, and tapers at the end. The schedule is parameterised by cumulative noise levels $\bar{\alpha}_t$: at $t = 0$, $\bar{\alpha}_0 = 1$ (clean image); at $t = T$, $\bar{\alpha}_T \approx 0$ (pure noise).

An interactive **timestep slider** (0–50) lets participants watch a single image progress from clean to noisy, with the schedule curve displayed alongside.

4.4 Noise Prediction Loss

The training objective is straightforward: given a noisy image at timestep t, the denoiser predicts the noise that was added. The loss is the mean squared error between the predicted noise and the true noise.

4.5 Productive Failure: The Naïve MLP

Participants first train a **fully connected network (MLP)** as the denoiser. The MLP flattens the image into a single vector, destroying all spatial structure.

The result: the MLP generates images indistinguishable from pure noise. It has learned nothing useful because it cannot exploit the two-dimensional structure of images.

Common Pitfall

A naïve MLP treats every pixel independently and has no notion of spatial neighbourhood. Flattening a 2D image into a 1D vector discards exactly the structure needed for coherent image generation.

Key Insight

The architecture of the denoiser determines *what kinds of structure* the model can learn. Choosing the right architecture is not a minor implementation detail—it is a strategic decision that shapes output quality. The fix: switch to a **convolutional** architecture (the U-Net, introduced in Notebook 4).

Real-World Story: The Deepfake Dilemma

In 2023, AI-generated images were submitted as legal evidence. They looked photorealistic because photorealism is the explicit *training objective*—the model minimises the difference between generated and real image distributions. Detecting AI-generated content requires specialised tools that exploit subtle frequency-domain artefacts left by the generation process.

Manager's Briefing

Denoising Steps = Cost. Each denoising step requires a full forward pass through the neural network. Fifty steps means fifty times the compute of a single image classification.

Setting	Steps
Standard quality	50
Production (balanced)	20–50
Fast / distilled models	1–4

Vendor questions: How many denoising steps does your model use? Is guidance computed

in parallel or serially? Can users trade quality for speed?

5 Notebook 4: The Conversation—Cross-Attention

Approx. 30 minutes. Metaphor: "At every turn, the driver asks the co-pilot."

5.1 From Unconditional to Conditional Generation

The diffusion model from Notebook 3 generates images, but it has no way to steer *what* it generates. The driver can navigate but does not know the destination. We need a mechanism for the text prompt (via CLIP embeddings) to influence the denoising process at every step.

5.2 The MicroUNet Architecture

The notebook introduces a small convolutional denoiser—the **MicroUNet** (∼160K parameters)—with the following structure:

1. **Encoder:** three convolutional layers with GroupNorm, progressively downsampling the 32×32 image to an 8×8 feature map (128 channels).
2. **Bottleneck:** timestep injection via a learned embedding, plus a **cross-attention layer** connecting image features to text features.
3. **Decoder:** two transposed convolutions with skip connections from the encoder, restoring the original resolution.

5.3 How Cross-Attention Works

Cross-attention is the mechanism by which image features "consult" text features during denoising. At the bottleneck of the U-Net:

1. The 64 image patches (8×8 spatial grid, 128-dimensional features) produce **queries** (Q): "What should I look like here?"
2. The per-token text embeddings (32-dimensional) produce **keys** (K) and **values** (V): "Here is what the prompt says."
3. Each query is compared to all keys via a scaled dot-product similarity.
4. The resulting **attention weights** determine how much each text token influences each spatial location.
5. The weighted values are projected back to 128 dimensions and added as a **residual connection** to the image features.

This process repeats at every denoising step, providing continuous text guidance.

Key Insight

Cross-attention is not a one-time instruction; it is a *continuous conversation* between the image being generated and the text prompt. At every denoising step and at every spatial location, the image "asks" the text what to do.

5.4 Demo: Cross-Attention ON vs. OFF

A side-by-side comparison shows the same model generating with cross-attention enabled (sharp, text-aligned shapes) versus disabled (blurry, generic output). The contrast is immediate and dramatic.

5.5 Visualising Attention Heatmaps

Attention weights are visualised as heatmaps over the 8×8 spatial grid, one heatmap per text token. Participants observe that the token "red" activates the spatial locations where the shape appears, while "circle" activates the shape's boundary region.

5.6 Interactive Explorer: Guidance Scale

A slider varies the **guidance scale** from 1.0 to 15.0, showing five side-by-side generations:

- **Low scale ($\sim$1):** the co-pilot whispers—creative but unfocused output.
- **Sweet spot (7–10):** the co-pilot speaks clearly—sharp, text-faithful results.
- **High scale (>10):** the co-pilot shouts—follows instructions obsessively but may oversaturate.

5.7 Productive Failure: Uniform Attention

When attention weights are forced to be *uniform* (every text token influences every pixel equally—"co-pilot with earplugs"), the output collapses into a blurry, generic blob that ignores the prompt entirely. Restoring the learned attention weights immediately recovers sharp, text-aligned generation.

Real-World Story: The E-Commerce Material Swap

A company prompted a model with "silver watch on wooden table." The output: a wooden-looking watch on a metallic table. This *attribute binding* failure arises because cross-attention has no built-in constraint forcing attributes ("silver," "wooden") to bind to the correct objects.

Manager's Briefing

Compositional Accuracy. Cross-attention enables text guidance but suffers from the *attribute binding problem*: when multiple objects and attributes are present, the model can swap which attribute belongs to which object. There is no hard architectural constraint preventing this.
Vendor question: "Can you show me attention maps for a failed generation?" Attention visualisation reveals *why* a model produced a wrong output—invaluable for debugging.

6 Notebook 5: The Road Trip—The Full Pipeline

Approx. 30 minutes. Metaphor: "Start the engine, set the destination, drive."

6.1 Assembling the Pipeline

In this final notebook, all components are wired together into a complete text-to-image system:

1. The **text prompt** is encoded by the trained Mini-CLIP text encoder into per-token embeddings.
2. A **random noise image** is sampled (determined by the seed).
3. Over 50 denoising steps, the **MicroUNet** progressively removes noise, consulting the text embedding via **cross-attention** at each step.
4. The final output is a clean, generated image matching the prompt.

The climactic moment of the course: participants type a prompt such as "red circle" and watch the system they built transform pure noise into a recognisable red circle, displayed as a filmstrip of intermediate denoising frames.

6.2 Classifier-Free Guidance

At each denoising step, the model runs **two forward passes**:

1. **Conditional:** the U-Net receives the actual text embedding.
2. **Unconditional:** the U-Net receives a null (empty) text embedding.

The two predictions are blended:

$$\hat{\epsilon} = \epsilon_{\text{uncond}} + s \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

where s is the guidance scale. A scale of $s = 1$ yields purely unconditional generation; values of $s = 7$–10 produce sharp, text-faithful images; very high values ($s > 15$) cause oversaturation and artefacts.

In the co-pilot metaphor: the guidance scale controls how loudly the co-pilot speaks. Too quiet and the driver wanders; too loud and the driver oversteers.

6.3 Interactive Prompt Lab

Participants experiment freely with:

- A **prompt dropdown** (all 9 base classes plus variations)
- A **seed slider** (0–999)
- A **guidance scale slider** (1.0–15.0)

Each generation displays a filmstrip of eight intermediate denoising frames, making the progressive refinement process visible.

6.4 Productive Failure: Compositional Complexity

The dataset is expanded from 9 classes to **36 classes** (adding two sizes and two positions). The model's colour and shape accuracy drops, especially when multiple attributes must bind correctly (e.g., "small red circle" sometimes produces a small circle with red appearing elsewhere).

This demonstrates the fundamental **combinatorial challenge**: the number of possible attribute combinations grows multiplicatively, but training data is finite. Real-world models face the same scaling problem at vastly larger scale.

6.5 Safety: Where Would Filters Go?

The notebook discusses where safety filters belong in the pipeline:

1. **Text input:** reject or rewrite blocked prompts before encoding.
2. **CLIP embedding:** check the embedding vector against known harmful regions.
3. **Final image:** classify the output and block unsafe content.

Because text guidance operates at *every* denoising step, a harmful prompt influences the entire generation process—making input-level filtering the most effective first line of defence.

6.6 Bridge to Real Tools

The toy model (∼300K parameters) and production Stable Diffusion (∼860M parameters, 512×512 output) share the *same architecture*: CLIP text encoder, U-Net denoiser with cross-attention, cosine noise schedule, and classifier-free guidance. The only difference is scale. SDXL pushes further to ∼3.5B parameters and 1024×1024 resolution. Understanding the toy system transfers directly to reasoning about production systems.

Real-World Story: The Prompt Engineering Economy

Small wording changes—"a red circle" versus "circle, red"—produce different CLIP embeddings and therefore different outputs. A marketing team discovered that switching from casual language to precise descriptors ("product photography of blue running sneaker, centred, white studio background. . . ") dramatically improved consistency and quality. Prompt sensitivity is an *architectural property*, not a user error.

Manager's Briefing

Final Briefing. You have now built every component of a text-to-image system at miniature scale. Real Stable Diffusion is roughly 25 000× larger but **architecturally identical**. When your engineering team says "increase the guidance scale," you know exactly what that means: two forward passes per step, blend the predictions, amplify the text signal. You understand the system *mechanically*, not vaguely.

7 The Co-Pilot Reference Card

Each notebook adds one row to a cumulative reference table. By the end of Notebook 5, participants have a complete cheat sheet:

NB	Component	What It Does	Co-Pilot Metaphor
1	Raw Ingredients	Images = tensors of numbers; text = embedding vectors; cosine similarity measures closeness	"Here are the parts of the car and the map"
2	CLIP (Translator)	Learns a shared embedding space; contrastive loss pulls matching pairs together	"The co-pilot learns to read the map"
3	Diffusion (Engine)	Forward: add noise. Reverse: predict & remove noise. U-Net preserves spatial structure	"The driver learns to navigate by being dropped at random locations"
4	Cross-Attention (Conversation)	Q = image, K/V = text. Each image region checks the text. Guidance scale = co-pilot volume	"At every turn, the driver asks the co-pilot"
5	Full Pipeline (Road Trip)	Wire it all together: noise → denoise with text guidance → image. CFG = two passes	"Start the engine, set the destination, drive"

8 Five Questions to Ask an AI Image Generation Vendor

1. **Resolution & cost** (from NB 1). What resolution does your model generate natively, and what is the per-image compute cost?
2. **Training data & bias** (from NB 2). How large and diverse is your training dataset? What bias audits have been conducted?
3. **Architecture & speed** (from NB 3). How many denoising steps does your model use? Can users trade quality for speed?
4. **Diagnostics** (from NB 4). Can you provide attention maps or other interpretability tools to diagnose failed generations?

5. **Reproducibility** (from NB 5). Does your system log random seeds and model versions for every generated asset, enabling deterministic replay?

9 Vendor Evaluation: Decision Tree

When adopting AI image generation, organisations face three broad options:

API Service
Use a third-party API (e.g., OpenAI, Stability AI). Best when: speed to market is paramount, volume is moderate, and customisation needs are low. Trade-off: the vendor controls model updates and data handling.

Fine-Tuned Model
Take a pre-trained open model and fine-tune it on proprietary data. Best when: domain-specific quality matters (e.g., your brand's visual style) and you have in-house ML capacity. Trade-off: faster convergence but requires curating training data.

Self-Hosted Model
Deploy a model entirely on your own infrastructure. Best when: data privacy is non-negotiable, you need full control, and you have significant engineering resources. Trade-off: you bear all compute cost and model maintenance.

The right choice depends on your organisation's constraints along four axes: **data sensitivity**, **customisation depth**, **engineering capacity**, and **cost structure**.

10 Jargon Decoder

Term	Plain-Language Definition
Tensor	A multi-dimensional grid of numbers (an image is a 3-D tensor)
Embedding	A list of numbers representing a word, sentence, or image in a form computers can process
Cosine similarity	A score measuring how similar two embeddings are (1 = identical direction, 0 = unrelated)
CLIP	A model that learns to place matching images and texts near each other in a shared number space
Contrastive learning	A training strategy that pushes matches together and mismatches apart
Diffusion	A generative process that creates images by iteratively removing noise from random static
U-Net	A convolutional neural network with skip connections, well suited for image denoising
Cross-attention	A mechanism allowing one type of data (images) to query another (text) for guidance
Guidance scale	A parameter controlling how strongly the text prompt influences image generation
Noise schedule	The plan governing how quickly noise is added or removed at each step
Latent space	A compressed numerical representation where images are manipulated more efficiently
VAE	Variational Auto-Encoder: compresses images into latent space and reconstructs them back
LoRA	Low-Rank Adaptation: a lightweight fine-tuning technique that modifies only a small fraction of model weights
Fine-tuning	Further training of an existing model on specialised data to improve domain-specific quality
Inpainting	Selectively regenerating a chosen region of an image while keeping the rest unchanged

11 Red Flags When Evaluating Vendors

1. **"We use a proprietary architecture."** If the vendor cannot describe their model's architecture, you cannot evaluate its limitations or plan for adaptation.
2. **No guidance scale parameter.** Without user-controllable guidance, you cannot tune the balance between creativity and prompt fidelity for your use case.
3. **Fixed single-step generation.** A system that does not expose the number of denoising steps removes a key quality–speed trade-off lever.
4. **No API for custom training.** If the vendor offers no path to fine-tuning, you are permanently locked into their base model's capabilities and biases.
5. **No bias or safety audit.** The vendor cannot describe what biases exist in their training data, what safety filters are in place, or how content provenance is tracked.

12 Conclusion

The core architecture of text-to-image generation rests on three learnable components—a text–image alignment model (CLIP), a convolutional denoising network (MicroUNet with diffusion), and a conditioning mechanism (cross-attention)—plus a set of carefully chosen hyper-parameters (cosine noise schedule, guidance scale, seed).

None of these pieces is, on its own, mysterious. Images are grids of numbers. Text is lists of numbers. CLIP aligns the two. Diffusion learns to remove noise. Cross-attention lets text steer the denoiser. Classifier-free guidance amplifies the text signal. The compound system produces results that feel like magic, but every step is auditable, tuneable, and—as you have now experienced first-hand—buildable from scratch.

Understanding these building blocks transforms the conversation with engineering teams and vendors from "Can the AI do this?" to "Which component is responsible for this limitation, and what would it take to fix it?"

These lecture notes accompany the interactive course "Stable Diffusion from Scratch: A Decision-Maker's Guide."
All interactive elements, code exercises, and pre-/post-course quizzes are available in the companion Jupyter notebooks.